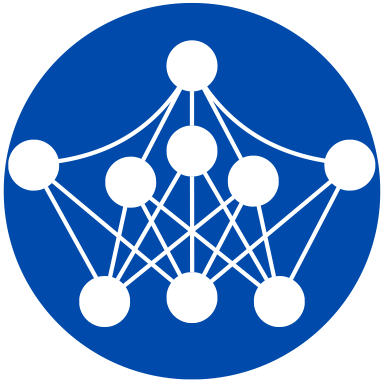
Oleksandr Akimenko

Machine Learning From Scratch



Intuition, Math and
Code of ML Algorithms

Machine Learning From Scratch

Intuition, Math and
Code of ML Algorithms

Oleksandr Akimenko

Contact: akm.oleksandr@gmail.com

Companion code: github.com/ml-from-scratch-book/code

ISBN: 978-1-0675197-1-1

Who Should Read This Book

This book is for two audiences: those with no machine learning (ML) experience and those seeking to deepen their understanding of core concepts. The only prerequisites are basic Python and high school math.

What Sets This Book Apart

This book is built around four core principles that guide how machine learning should be understood and applied: depth, simplicity, structure, and relevance.

Depth

The only way to truly understand how something works is to build it from scratch. Grasping how algorithms learn and make predictions at a fundamental level gives you the clarity to know when and how to apply them successfully in practice.

Simplicity

Machine learning has a reputation for being inaccessible. Many believe a STEM degree plus a master's or PhD is required to truly understand it – and gatekeeping within the community only reinforces that belief. This book exists to change that.

Understanding the basics of linear algebra, calculus, statistics, and computer science is necessary to fully grasp ML, but it doesn't have to be hard. The biggest obstacle isn't the math itself – it's that people give up before discovering its practical value. This book takes a pragmatic approach, presenting only the math needed to build real ML algorithms from scratch, letting these concepts sink in naturally through implementation.

Computer science here serves as a tool to formalize theory. Every concept is followed by a Python implementation – think of it as building a house once you have the blueprint, materials, and proportions.

Structure

There are many good individual tutorials about various ML concepts that exist online. However, building a proper foundation requires understanding how different concepts relate to each other. The connections between concepts are the key to clarity. This book builds up complexity incrementally with each ML section following the exact same logical structure.

Relevance

The concepts in this book are absolute essentials for both day-to-day ML work and acing ML theory interviews at any level. The industry recognizes these algorithms as the true foundation.

These four qualities together enable people with no formal technical background to fully understand and leverage core machine learning concepts to solve real problems.

This insight comes from an author who started learning machine learning with nothing more than high school math and basic Python – no formal university education at the time. Working through countless online lectures, courses, and tutorials from leading institutions, the process often felt fragmented and frustrating. That experience made one thing clear: this book needed to exist.

Years later – after completing relevant university courses, being mentored by industry veterans at top companies, and gaining hands-on experience at TD Bank, Kenvue (formerly Johnson & Johnson), and multiple startups – this book is in your hands.

Table of Contents

Setup

To get the most out of this book, it is strongly recommended that you implement the machine learning algorithms as you read. All examples in this book are designed to be run in Jupyter Notebooks. Unlike Python files, Jupyter Notebooks preserve code outputs alongside the code itself, which is useful for machine learning development.

There are multiple ways to set up your environment depending on your system and preferences. While this book assumes you are using Jupyter Notebooks, you are welcome to use any Python development environment you're comfortable with.

Running the Code without Local Installation

If you want to get started immediately without installing anything locally, you can use Google Colab throughout the entire book. Google Colab runs in the browser and works on any Mac or PC with Google Chrome installed. It is a simple and reliable way to run all the codes present in this book.

Here is the detailed instruction on how to get started:

1. Open Google Chrome and navigate to Google Drive (drive.google.com)
2. Create a new folder to store all the code for this book
3. Inside the folder: Right click, navigate to "More" and select "Google Colaboratory" – this creates a new Jupyter notebook
4. Start coding

Required Python Libraries

To run the examples in this book, several Python libraries are required. If you are using a local Python environment, these libraries need to be installed just once. Google Colab has most libraries pre-installed but those that are not must be downloaded inside each notebook that uses them.

Here's the list of all libraries that will be used throughout the book: NumPy, SciPy, Pandas, Scikit-learn, XGBoost, PyTorch, Optuna, OpenML.

If you already have a Python environment running locally, you can install all of them in the terminal using the following line:

```
pip install numpy scipy pandas scikit-learn xgboost torch optuna openml
```

If you choose to use Google Colab, most libraries are already pre-installed. However, whenever Optuna or OpenML are needed, you can install them with this line of code:

```
!pip install optuna openml
```

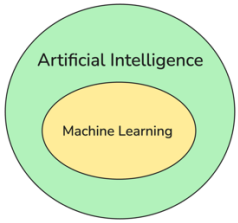
All substantial code examples are available in the GitHub repository at: **github.com/ml-from-scratch-book/code**

Fundamentals of ML

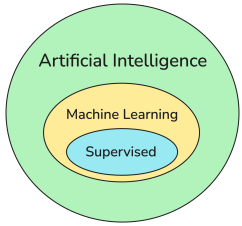
We will begin with some fundamental definitions you need to get comfortable with.

- Algorithm – a set of instructions that need to be followed in order.
- Artificial Intelligence (AI) – simulation of human intelligence in machines.
- Machine Learning (ML) – a subset of AI concerned with the development of algorithms that can learn from data and generalize to unseen data. Thus, ML algorithms can perform tasks without explicit instructions.

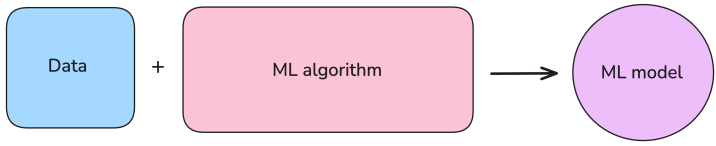
Most of the time when people talk about artificial intelligence, they refer to machine learning.



While there are multiple types of machine learning, supervised learning is the most practical. Most AI systems today run on supervised algorithms, and other approaches build heavily on its techniques. Therefore, all algorithms covered here focus on supervised machine learning, giving you skills that transfer to any ML subfield.



ML model – the result of training an ML algorithm on data.



An ML model can be thought of as a student, who learns material from a book and then applies it in the real world. An ML professional who is developing that algorithm is a teacher.

Our Job as Teachers

As teachers, our goal is to create the ideal learning environment and provide the student with the right resources to master certain concepts. This comes down to selecting a relevant book that matches how that student learns best and giving them time to prepare.

In the end, the teacher tests the student on new problems that require understanding of the desired concepts. The goal of the test is to assess how well the student has learned information of interest. In other words, simulate how well they will solve desired problems when they leave the classroom and enter the real world.

Our goal is for the student to succeed in the real world. Hence, we train them until their performance on unseen problems – the test is sufficient.

Now, re-read this analogy by replacing the teacher with an ML professional (you), the student with an ML algorithm and the book with data.

Following this analogy, from our perspective ML algorithms have two modes:

1. Training – the learning process of the algorithm. In other words, the process of discovering general patterns in the data that apply to most problems of this kind.
2. Inference – leveraging learned patterns to make predictions on unseen data.

An important distinction that needs to be made is between seen and unseen data. Data is considered seen when it was part of the dataset the ML model was trained on – just like problems in the book a student used to prepare for the test. Conversely,

data is considered unseen when the model wasn't trained on that data – new problems of similar kind the student faces on a test or when they enter the real-world.

As a professional we want the ML model to learn general patterns in the data. General patterns are those that are applicable to many problems of this kind – not just a single instance of it.

If a model succeeds in learning general patterns – it is said to have a right fit to the data. If not, there are two options for what could have happened:

1. Underfit – the model was unable to learn patterns because they are too complex, conversely the model is too simple
2. Overfit – the model memorized the data but didn't capture the general patterns

Both cases mean that the model will not be able to solve similar problems that were not previously seen.

Our goal as professionals is to balance the learning ability and memorization, so that our ML model learns only important patterns. Let's illustrate this with a continued student example.

Imagine these are the problems from the book our student was trained on:

1. John has 3 apples, Alisa has 7 apples and Dan has 2 apples. How many apples is there in total? The answer: 12
2. Anna has 4 apples, Robin has 7 apples and Li has 8 apples. How many apples is there in total? The answer: 19

Formally, a student learns in the following way:

1. Reads the problem description and solves the problem using their current knowledge
2. Compares their solution with the answer and tailors their approach if solution was incorrect
3. Repeats 1 and 2 for every problem in the book

Once the student was trained, we test them on a similar problem they haven't seen yet to observe how well they learned. For example, the following problem:

Gabe has 1 apple, Robin has 7 apples and Jane has 8 apples. How many apples is there in total? The answer: 16 but the student was not provided with the answer.

Let's explore three possibilities of what the student could have learned.

The Right Fit

The student learned the correct pattern. An example of the student's thinking: "I see the pattern! To find the total number of apples, you just need to add up the number of apples each person has. Let's apply it to the new problem. The total number of apples is 1 + 7 + 8 = 16."

In this case, the student identified the correct, general rule that works for both the training problems and the new, unseen problem. If we measure performance as "good" or "bad", we can conclude the following result:

- Seen data: good
- Unseen data: good

Underfit

The student learned an overly simple rule. An example of the student's thinking: "Maybe the answer is always just the number of apples the first person has? In the first problem, John had 3 (close-ish to 12?). In the second, Ana had 4 (close-ish to 19?). Let's apply this rule on the new problem. The first person is Gabe, who has 1 apple, so the total number of apples is 1."

In this case the student hasn't learned how to solve problems correctly (couldn't even match the answers in the book). Hence, no problems were solved, and the resulted performance is:

- Seen data: bad
- Unseen data: bad

Overfit

The student memorized unimportant details. An example of student's thinking: "Maybe certain names map to a particular number of apples in total. The first problem features name "Alisa" and the answer is 12. The second problem features name "Robin" and the answer is 19. Let's apply this rule on the new problem. I remember

that second problem! It featured "Robin", and the answer was 19. This new problem also has "Robin" in it, so the answer must be 19 again!"

In this case, the student didn't learn the general concept. Instead, they fixated on a specific detail from the training data – "Robin" appeared in the problem with answer 19 and the student treated it as the rule. It works for that one example but fails to generalize.

- Seen data: good
- Unseen data: bad

Once a model is trained, we can figure out whether it has the right fit, underfits or overfits the data by looking at the performance on both seen and unseen data. To conclude:

- The right fit – seen: good, unseen: good. It is exactly what we want!
- Underfit – seen: bad, unseen: bad. The model is too simple, meaning it can't learn any pattern to produce a remotely correct answer even on seen data. Hence, we choose a more complex ML algorithm.
- Overfit – seen: good, unseen: bad. The model is too complex because it memorizes unimportant information. Hence, we choose a simpler ML algorithm.

Since there's no scenario when a model has bad performance on seen and good performance on unseen data – these three scenarios cover all possible cases.

Conclude: Our ultimate goal is to maximize performance of the model on unseen data as it mirrors model's performance in the real-world. By observing how the model performs on seen and unseen data – we can conclude whether it's the right fit, underfits or overfits the data to inform our further actions.

Let's now illustrate how problem in the example could be solved with an ML algorithm. Unlike a student, an ML algorithm needs a more structured representation of the information in order to learn. Hence, we convert problem from the example into a data table.

Training data looks as follows:

Name_1	Name_2	Name_3	Apples_1	Apples_2	Apples_3	Total
John	Alisa	Dan	3	7	2	12
Anna	Robin	Li	4	7	8	19

Structured data is usually represented by a table where every row corresponds to a separate observation and every column corresponds to a certain characteristic of every observation. In this case each problem corresponds to a row. Just like for a student, we need to explicitly indicate what ML model needs to learn. In this case it's the answer to a problem – column with the name "Total".

In every problem, the description and the answer must be treated separately. Hence, we give each of them a name and notation that applies to all ML problems.

- Features – a general name for a structured problem description in ML, denoted by X
- Target – a general name for an answer column, denoted by y

By following this convention, we can represent training data as follows:

Features – X:

Name_1	Name_2	Name_3	Apples_1	Apples_2	Apples_3
John	Alisa	Dan	3	7	2
Anna	Robin	Li	4	7	8

Target – y :

Total
12
19

Once data is ready, it's time to train an ML model. The goal is for the model to be capable of solving this kind of problems independently. The general idea behind the learning process of an ML model is to learn a rule (or set of rules) that maps features X to the target y. Thus, we conclude that during the training process model receives both X and y as inputs. Then it proceeds to learn general rule(s) of how to predict y based on X. In other words, infer answers based on problem descriptions.

Once the model was trained, we test it on unseen problems of similar kind. Thus, we prepare a new problem description of a similar kind in the same structured way.

Features – X:

Name_1	Name_2	Name_3	Apples_1	Apples_2	Apples_3
Gabe	Robin	Jane	1	7	8

The ML model takes features as input and predicts the target based on the patterns it learned during training. Recall, this process is also referred to as inference.

Once the prediction is made, we compare it with the true answer to measure how accurate the prediction is – just like testing a student. In case performance on unseen data is good – the fit is just right, and our model is ready to independently solve problems of this kind in the real-world.

If performance is not good, we can check whether the model underfits or overfits the data by assessing performance on seen data. This informs us of whether we should increase or decrease complexity of the model or select a new one.

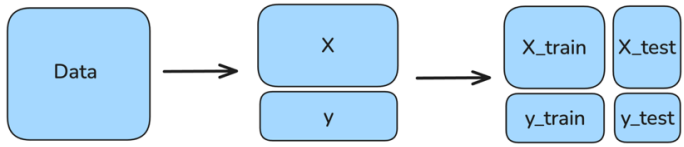
In the real world we usually get a single dataset. Therefore, to train and test an ML model we need to split the data in two subsets. Seen data is also called training data because that's what we train our ML model on. Unseen data is also called test data because it is used to assess model performance to understand how well it will perform in the real world.

The proportion of the split depends on size of the dataset and the problem but usually we allocate 80% of rows for training and 20% for testing.

Let's formalize end to end ML project stages that we loosely defined so far:

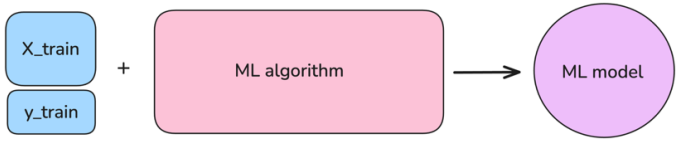
Data Preparation

First, we have data in a table format, then we split it into features and targets, X and y respectively. Then we split it further for training and testing. As a result, our initial dataset is split into four subsets, two for training: X_train, y_train and two for testing: X_test, y_test:



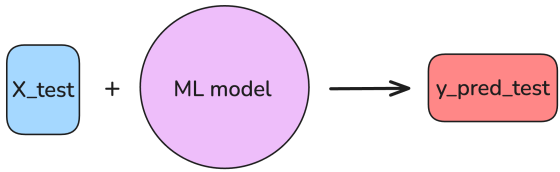
Training

During training, we pass X_train, y_train pairs as input into the ML algorithm to get a trained ML model.



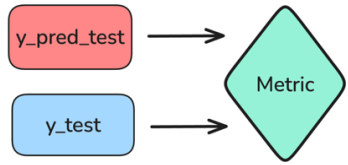
Testing

During testing, the model performs inference on X_test to make predictions.



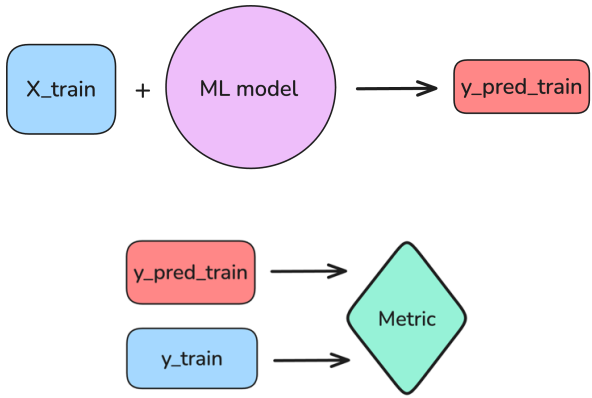
Then we compare y_pred_test with y_test to see how accurate the model's predictions are. This gives us insight into how well the model can deal with problems it hasn't seen during training.

The performance is measured by some metric that tells us how far *y_pred_test* is from *y_test*.



Based on the performance of the model on unseen data we decide whether the current model is good enough for our application. If we are satisfied – we proceed to use this model to solve our real-world problems. Note that using a model is equivalent to making the model perform inference on the new data of interest.

Otherwise, we measure the model's performance on seen data and decide whether our model underfits or overfits.



This tells us whether the model is too simple or too complex. With this insight, we gain more intuition about the nature of this problem, which helps us decide whether to adjust the model's complexity or try a different algorithm.

Congratulations on learning core ML definitions and the fundamental ML project structure. In this module we conceptually went from having a dataset to training and testing an ML model.

Introduction to Data

In the previous section we explored the core principles of ML by going over a very simple problem. It has done a perfect job playing along with the student analogy and showcasing general concepts in ML. However, it is too simple of a problem – in the real-world we would not bother using ML to solve an addition problem. In fact, if a problem can be solved sufficiently well with a deterministic method, there's no need to use ML. In this chapter we will get some feel to what real-world data looks like and how to deal with it.

How are problems and data in the real world different from the student example?

- Bigger and messier datasets. We could never train a good model on just two observations. In the real world it is common for datasets to consist of thousands or millions of observations. Additionally, in the student example we had complete and correct problem descriptions, which is not necessarily the case in the real world.
- More complex problems. In the student example we knew the perfect way to map features to targets – addition. In reality, we should only bother with ML if we don't know it. The goal of an ML model is to learn an equation or set of rules that allow it to predict target based on features.

An example of a real-world ML problem:

Predict the median house value in various California districts based on a range of features, which include demographic data and housing characteristics. Our goal is to train a model capable of accurately predicting median house value in a neighborhood.

Below is a snapshot of this dataset:

	median_income	housing_median_age	total_rooms	...	latitude	longitude	median_house_value
0	8.3252	41	880.0	...	37.88	-122.23	452600.0
1	8.3014	-99	NaN	...	37.86	-122.22	358500.0
2	7.2574	52	1467.0	...	37.85	-122.24	352100.0
3	0.0000	52	1274.0	...	37.85	-122.25	341300.0
4	3.8462	52	1627.0	...	37.85	-122.25	342200.0
...	...	...	...	...	...	...	...
20635	NaN	25	1665.0	...	39.48	-121.09	78100.0
20636	NaN	18	697.0	...	39.49	-121.21	77100.0
20637	NaN	17	2254.0	...	39.43	-121.22	92300.0
20638	NaN	18	1860.0	...	39.43	-121.32	84700.0
20639	NaN	16	2785.0	...	39.37	-121.24	89400.0

20640 rows × 9 columns

Let's explore the columns one by one. Note that the screenshot shows a subset of the data for illustration – you will work with the complete datasets in upcoming chapters.

Features:

- median_income – median income in the district (in tens of thousands of dollars)
- housing_median_age – median age of houses in the district
- total_rooms – the total number of rooms in the district
- total_bedrooms – the total number of bedrooms in the district
- population – the total population in the district
- households – the total number of households in the district
- latitude – latitude of the district (location data)
- longitude – longitude of the district (location data)

The target:

- median_house_value – median house value (price in dollars)

Let's see how differences between the student example and real-world problems manifest in this case.

As you can observe, this dataset contains 20,640 rows and 9 columns. Additionally, it has quality issues typical of real-world data: some values are missing (NaN), while others are impossible – such as the median income of $0 at cell (3,0) or the house age of -99 at cell (1,1). These errors arise from various sources: manual data entry mistakes, bugs in data collection systems, sensor malfunctions, etc.

The most important difference is that it would be difficult for a human to manually define an equation or a fixed set of rules to accurately relate features to the target variable. Therefore, we need ML to solve this problem.

Now that we identified points of difference – can we just split this data, train and test the model as outlined in the previous chapter?

Theoretically yes, but that would be like giving a student a textbook full of typos and missing pages.

In the previous section, it was mentioned that it's the teacher's job to provide the student with learning materials that best suit them. In other words, learning materials that make the student learn in the most effective way and thus maximize their performance on a fair test. After all, a student can learn as well as the learning materials teach them.

Additionally, different students learn best with different teaching approaches. Therefore, a teacher needs to truly know how the student thinks in order to provide them with the best suited learning materials.

This all makes sense, but how does it translate to machine learning?

As ML professionals, we're typically given raw data. Data preparation and algorithm selection are our responsibility. As you have already seen, raw datasets can contain errors and missing data. Training a model on that data is equivalent to teaching a student with problem descriptions that have missing information and errors. This leads both an ML model and a student to learn these mistakes, which we want to avoid. Therefore, always remember – we have complete freedom to transform the data however we want.

Different ML algorithms are like different students – they benefit from different learning approaches. When we understand how an algorithm learns (which you will discover in this book), we can prepare data that leverages its full potential. This maximizes the model's performance on unseen data, which reflects its real-world effectiveness.

How do we make the dataset ready for an ML algorithm?

Data preprocessing can be seen as a two-step process: Data Cleaning and Feature Engineering. Before diving in, there's one rule that governs both steps – every preprocessing operation falls into one of two types:

- Row-independent – the operation only looks at a single row to compute a value. These are safe to apply at any time, so we apply them on the whole dataset before the split. These tend to be column-wise operations.
- Row-dependent — the operation looks across multiple rows to compute a value. Consider why it must be applied after the split: if we use all data to compute a value, the test set is no longer truly unseen – its information has already quietly shaped our training set. This means our model's performance on the test set no longer reflects how it would perform in the real world because we wouldn't have access to unseen data there.

You will see an example of each operation type as we dive into the two steps of data preprocessing.

Data Cleaning

In this step, we handle missing values and errors by either deleting rows and columns or finding reasonable estimates.

If a row or column has many missing values, the simplest solution is to remove it. For example, we will remove 2nd row and 1st column of our dataset to get:

	housing_median_age	total_rooms	total_bedrooms	...	latitude	longitude	median_house_value
0	41	880.0	129.0	...	37.88	-122.23	452600.0
2	52	1467.0	190.0	...	37.85	-122.24	352100.0
3	52	1274.0	235.0	...	37.85	-122.25	341300.0
4	52	1627.0	280.0	...	37.85	-122.25	342200.0
5	52	919.0	213.0	...	37.85	-122.25	269700.0
...	...	...	...	...	...	...	...
20635	25	1665.0	374.0	...	39.48	-121.09	78100.0
20636	18	697.0	150.0	...	39.49	-121.21	77100.0
20637	17	2254.0	485.0	...	39.43	-121.22	92300.0
20638	18	1860.0	409.0	...	39.43	-121.32	84700.0
20639	16	2785.0	616.0	...	39.37	-121.24	89400.0

20639 rows × 8 columns

However, when a column is crucial and the dataset isn't large enough to freely delete rows, we need to estimate the missing values. Let's say we had kept "median_income" – what would be a reasonable estimate for its missing values?

A common approach is to fill the gap with an aggregated value – such as the mean "median_income" across all districts. We can also be more targeted: if location strongly influences income, we might use the mean income of the 10 nearest districts instead. This approach is called imputation of missing values.

Since it's a row-dependent operation, we will apply it after the split. For a simple imputation our plan is to compute mean "median_income" across all districts using training data. Then fill missing values in training and test data with the derived estimate.

Once our cleaning strategies are defined, we proceed to the next step.

Feature Engineering

This process involves transforming features to make patterns more obvious to the ML algorithm, just like simplifying problem descriptions for a student. The goal is to provide helpful hints without revealing the answer. Ask yourself: what indicators would make predicting house value easier?

A useful indicator could be ratio of the population to the total number of bedrooms. It reveals whether houses typically operate at full capacity or how "crowded" they are, which could be a strong indicator of a home's value.

We will call this new feature "used_capacity" and it is derived by dividing "population" by "total_bedrooms". The updated dataset looks as follows:

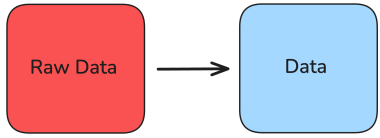
	housing_median_age	total_rooms	total_bedrooms	...	longitude	median_house_value	used_capacity
0	41	880.0	129.0	...	-122.23	452600.0	2.496124
2	52	1467.0	190.0	...	-122.24	352100.0	2.610526
3	52	1274.0	235.0	...	-122.25	341300.0	2.374468
4	52	1627.0	280.0	...	-122.25	342200.0	2.017857
5	52	919.0	213.0	...	-122.25	269700.0	1.938967
...	...	...	...	...	...	...	...
20635	25	1665.0	374.0	...	-121.09	78100.0	2.259358
20636	18	697.0	150.0	...	-121.21	77100.0	2.373333
20637	17	2254.0	485.0	...	-121.22	92300.0	2.076289
20638	18	1860.0	409.0	...	-121.32	84700.0	1.811736
20639	16	2785.0	616.0	...	-121.24	89400.0	2.251623

20639 rows × 9 columns

Since this operation is row-independent, we can apply it right away.

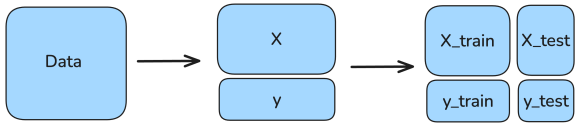
By creating features like this, we help the ML algorithm discover patterns that might otherwise remain hidden. We will develop much stronger intuition for feature engineering as we explore different ML algorithms. But you should remember that this step is more art than science. The better you understand how ML algorithms learn – the better your instincts will be for cleaning and engineering features that maximize performance.

Now that we have a general idea about data preprocessing, let's add this step to our ML project flow defined in the previous chapter:



In this context, "Raw Data" is the initial dataset, while "Data" represents a state where all row-independent cleaning and feature engineering operations have already been applied. Any row-dependent operations – like calculating mean across rows to fill missing values are planned at this stage but not yet executed.

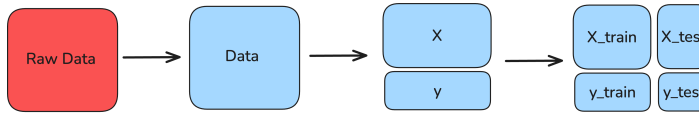
As the final data preparation step, the data is split – aggregations are then computed with just training data and used to transform both training and test sets.



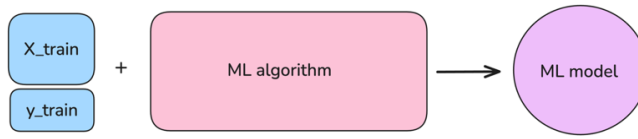
> Note: Though not shown in the diagram, this is the moment those planned row-dependent operations are executed – keep that in mind as you read forward.

With this new addition let's recap our finalized ML project flow at this stage:

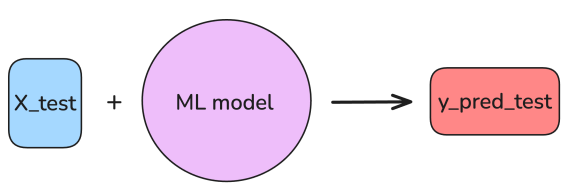
Data Preparation

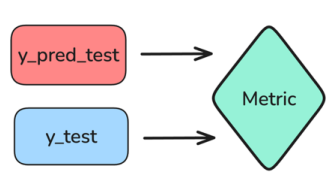


Training

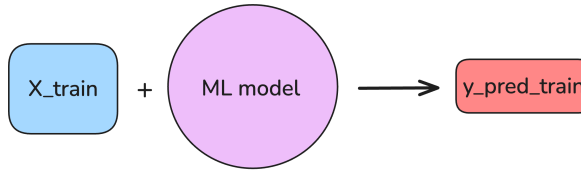


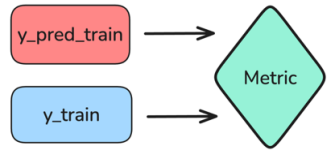
Testing





If we are satisfied with the performance – great, our model development is done! Otherwise, we check if the model underfits or overfits.





If training performance is still poor – the model underfits, meaning it's too simple to capture the underlying patterns in the data. If training performance is good or significantly better than test performance – the model overfits, meaning it's too complex and has memorized noise and irrelevant details instead of learning generalizable patterns.

Understanding how our ML algorithm learns from the preprocessed data allows us to reconsider both data preprocessing and algorithm selection. Perhaps our features need re-engineering to match the algorithm's learning style, or perhaps we need a different algorithm that better suits our current feature representation. These choices are interconnected – changing one often means reconsidering the other. Throughout this book you will learn how various algorithms learn and thus gain intuition on how to optimally leverage this information.

The Math You Actually Need for ML

Disclaimer: This chapter covers the practical math needed to build ML algorithms from scratch. While not as rigorous as a university math course, the information presented is accurate and sufficient for our purposes.

Manipulating Data (Linear Algebra Essentials)

This module will teach you all necessary data representations and operations to successfully navigate through theory and implementation of ML algorithms. Note that this module is not about strict mathematical concepts but rather practical application of linear algebra in programming. For convenience, we will revise all necessary concepts by employing NumPy – Python library for efficient computations. Let's begin by importing the necessary library:

```
import numpy as np
```

Importing "numpy as np", allows us to refer to the library by using just two letters instead of the full name – it's a standard convention for this library.

Generally, there are three primary ways to represent numeric data:

- Scalar – a single number. For example:

$$3$$

- Vector – a collection of numbers in one-dimensional list-like format. For example:

$$[1, 2, 3]$$

  The same information can be represented as:

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

- Matrix – a collection of numbers in two-dimensional table-like format. For example:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

The most appropriate representation depends on the context. For instance, if we want to represent the total number of people on Earth, a scalar makes sense. If we need the population of each country individually, we naturally need a vector. Now suppose we want to store both the population and average height for each country – a scalar won't work, and a vector becomes inconvenient. This problem naturally suggests a matrix, where each row corresponds to a country and each column corresponds to a different aspect of data – population, height, etc.

Each data structure supports specific operations. Just as we can add, subtract, multiply, and divide scalars (except division by zero), vectors and matrices have their own operations with corresponding rules.

To learn what operations you can perform on each one of these structures is to learn all linear algebra you need to understand everything about ML algorithms covered in this book.

In Python we can define a scalar, vector and a matrix in the following way:

```
scalar = 1

vector = np.array([1, 2, 3])

matrix = np.array([[1, 2, 3], [4, 5, 6]])
```

Next, we will perform some operations so note down the vector and the matrix to follow along.

Operations between vectors and scalars work intuitively – the scalar operation is applied to each element of the vector individually.

```
print(vector * 3)

[3 6 9]

print(vector - 3)

[-2 -1  0]

print(vector + 3)

[4 5 6]

print(vector / 3)

[0.33333333 0.66666667 1.        ]

print(np.power(vector, 3))

[ 1  8 27]
```

This approach is equivalent to iterating over a Python list and performing operations on each element individually. Though NumPy is much more efficient because it performs computations in parallel

Standard operations between two vectors of the same size work element-wise – each operation occurs between elements at corresponding positions.

```
print(vector * vector)

[1 4 9]

print(vector - vector)

[0 0 0]

print(vector + vector)

[2 4 6]

print(vector / vector)

[1. 1. 1.]

print(np.power(vector, vector))

[ 1  4 27]
```

There are two more useful operations we can perform with vectors in NumPy. The first one is comparing each element in a vector to a scalar or each element in another vector.

```
print(vector > 3)

[False False False]

another_vector = np.array([4, 5, 6])

print(vector < another_vector)

[ True  True  True]
```

The second operation is sum of all elements in a vector.

```
print(np.sum(vector))

6
```

Now we get to matrices. All the standard operations between scalars and matrices work as expected – simply perform the operation elementwise.

```
print(matrix * 3)

[[ 3  6  9]
 [12 15 18]]

print(matrix - 3)

[[-2 -1  0]
 [ 1  2  3]]

print(matrix + 3)

[[4 5 6]
 [7 8 9]]

print(matrix / 3)

[[0.33333333 0.66666667 1.        ]
 [1.33333333 1.66666667 2.        ]]

print(np.power(matrix, 3))

[[  1   8  27]
 [ 64 125 216]]
```

Performing any standard operation between a vector and a matrix applies the vector to each row of the matrix individually. Therefore, the operation is only valid if the vector length matches the number of matrix rows.

```
print(matrix * vector)

[[ 1  4  9]
 [ 4 10 18]]

print(matrix - vector)

[[0 0 0]
 [3 3 3]]

print(matrix + vector)

[[2 4 6]
 [5 7 9]]

print(matrix / vector)

[[1.  1.  1. ]
 [4.  2.5 2. ]]

print(np.power(matrix, vector))

[[  1   4  27]
 [  4  25 216]]
```

We can also perform all the standard operation between two matrices of the same size – ones with a matching number of rows and columns.

```
print(matrix * matrix)

[[ 1  4  9]
 [16 25 36]]

print(matrix - matrix)

[[0 0 0]
 [0 0 0]]

print(matrix + matrix)

[[ 2  4  6]
 [ 8 10 12]]
```

```
print(matrix / matrix)

[[1. 1. 1.]
 [1. 1. 1.]]

print(np.power(matrix, matrix))

[[    1     4    27]
 [  256  3125 46656]]
```

Similarly to a vector, we can sum up elements in a matrix. Since it's a two-dimensional structure, we can do it in three ways – sum up all columns, all rows or all elements. Below is shown how to perform each of these operations in respective order.

```
print(np.sum(matrix, axis=1))

[ 6 15]

print(np.sum(matrix, axis=0))

[5 7 9]

print(np.sum(matrix))

21
```

The last and the most important operation that we will cover in this chapter is matrix multiplication. Before exploring how this operation works, we first establish the conditions under which matrix multiplication is defined and how to determine shape of the resulting matrix. We begin by formally defining shape of the structure.

Shape of a matrix is defined by its number of rows and columns. A matrix with n rows and m columns is said to be of $n \times m$ shape, which reads as "n by m matrix". A vector can be viewed as a special case of a matrix in which one of the dimensions is equal to 1. A vector with k elements can therefore be represented as either $1 \times k$ (row vector) or $k \times 1$ (column vector). The relative order of vector elements stays the same regardless of whether the vector is written as a row or a column; however, their orientation matters in matrix multiplication. A scalar may be thought of as a 1×1 matrix. However, since scalars have no directional structure, matrix multiplication is not defined for scalar values.

Matrix multiplication is defined only when the number of columns in the first matrix matches the number of rows in the second matrix. The order of the operands matters.

Conceptually, we can say that matrix multiplication is only defined when the first matrix is of $a \times b$ shape and the second is of $b \times c$ shape. We can always find out shape of the resulting matrix based on shapes of two operands:

$$a \times b \cdot b \times c = a \times c$$

In other words, the resulting matrix has the same number of rows as the first matrix, and the same number of columns as the second matrix.

For example, consider two matrices: $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$. Both matrices are of 2×2 shape. Since the number of columns in the first matrix equals the number of rows in the second, the multiplication is valid. The resulting matrix will also have shape 2×2.

To compute the product of these two matrices, each entry of the resulting matrix is obtained as follows:

1. Take a row from the first matrix and a column from the second matrix
2. Multiply corresponding elements
3. Sum the products
4. Place the result in the corresponding position of the output matrix

This process is repeated for every row of the first matrix and every column of the second matrix. Elements of the resulting matrix are filled from left to right, top to bottom. To perform matrix multiplication, we follow the steps outlined above for all rows and columns of the matrices.

Here's the full process for two matrices above:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \text{ and } \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}, \text{ then } \begin{bmatrix} 1*5+2*7 & \\ & \end{bmatrix} = \begin{bmatrix} 19 & \\ & \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \text{ and } \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}, \text{ then } \begin{bmatrix} 19 & 1*6+2*8 \\ & \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ & \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \text{ and } \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}, \text{ then } \begin{bmatrix} 19 & 22 \\ 3*5+4*7 & \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & \end{bmatrix}$$

$\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix}$ and $\begin{bmatrix}5 & 6\\7 & 8\end{bmatrix}$, then $\begin{bmatrix}19 & 22\\43 & 3*6+4*8\end{bmatrix} = \begin{bmatrix}19 & 22\\43 & 50\end{bmatrix}$

The result looks as follows:

$$\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix} \cdot \begin{bmatrix}5 & 6\\7 & 8\end{bmatrix} = \begin{bmatrix}1*5+2*7 & 1*6+2*8\\3*5+4*7 & 3*6+4*8\end{bmatrix} = \begin{bmatrix}19 & 22\\43 & 50\end{bmatrix}$$

Recall, shape of operands helps you check validity of the operation and get to know shape of the resulting matrix.

Examples of valid matrix multiplications:

$$1 \times 5 \cdot 5 \times 10 = 1 \times 10$$

$$3 \times 3 \cdot 3 \times 5 = 3 \times 5$$

$$1 \times 10 \cdot 10 \times 1 = 1 \times 1$$

Examples of invalid matrix multiplications:

$$2 \times 2 \cdot 3 \times 2$$

$$10 \times 2 \cdot 10 \times 2$$

$$2 \times 2 \cdot 10 \times 2$$

Matrix multiplication between two vectors (arranged as row vector and column vector) produces a scalar, commonly called the dot product.

For example:

$$[1,2,3] \cdot \begin{bmatrix}4\\5\\6\end{bmatrix} = 1*4+2*5+3*6 = 32$$

Now let's explore matrix multiplication in Python. First, we will define matrices used in the example and verify our calculation.

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print(np.dot(A, B))

[[19 22]
 [43 50]]
```

Let's do the same with vectors used in the example.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.dot(a, b))

32
```

An alternative notation to perform matrix multiplication is as follows.

```
print(A @ B)

[[19 22]
 [43 50]]

print(a @ b)

32
```

NumPy treats one-dimensional arrays flexibly during matrix multiplication, automatically inferring whether to treat them as row or column vectors based on context to make the operation valid.

Since a vector is theoretically a special case of a matrix, we can also perform matrix multiplication between a vector and a matrix. For such operation to be valid in NumPy, either row or column form of the vector must be suitable.

For instance, let's explore $\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix}$ of shape 2×3 and $[1, 2]$ that can either by written in 1×2 or 2×1 format. How can we make this work? Well, the only arrangement that works shape-wise is:

$$1 \times 2 \cdot 2 \times 3 = 1 \times 3$$

Therefore:

$$[1, 2] \cdot \begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix} = [1 * 1 + 2 * 4, \quad 1 * 2 + 2 * 5, \quad 1 * 3 + 2 * 6]$$

$$= [9, 12, 15]$$

We can verify this in code below.

```
C = np.array([[1, 2, 3], [4, 5, 6]])
c = np.array([1, 2])

print (c @ C)

[ 9 12 15]
```

Sometimes, it's useful to change the shape of a matrix in order to enable matrix multiplication. This transformation is called transpose, and it converts columns of the matrix into rows and vise-versa. The transpose operation is denoted by the superscript T:

$$\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix}^T = \begin{bmatrix}1 & 4\\2 & 5\\3 & 6\end{bmatrix}$$

This matrix went from 2×3 to 3×2 shape. Transpose swaps rows and columns and thus it also swaps the numbers in the shape of a matrix.

Generally, if we apply transpose on a n by m matrix it becomes m by n. Let's now perform the transpose operation in Python.

```
print(C.T)

[[1 4]
 [2 5]
 [3 6]]
```

This concludes linear algebra and NumPy content required to fully understand core ML algorithms.

Understanding Data (Statistical Foundations)

Statistics is useful to answer questions about data – let's explore its application through a practical example.

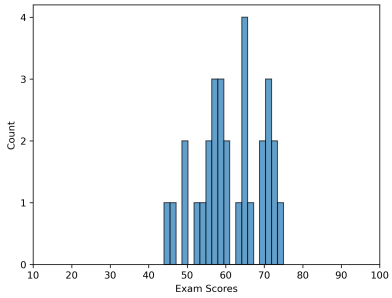
Grades for a university exam have just been released and we want to understand:

1. How well did the class of 30 students learn the material?
2. Did the professor do a good job teaching this class?
3. What grade should a future student in this class expect?

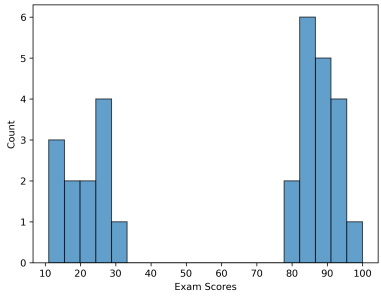
Intuitively, we want to see how well students performed on average, so we compute average exam score in this class, which turns out to be around 61. What can we conclude? On average, students tend to pass the class, which seems positive. But can we determine how good of a job the teacher did solely based on the average score? Not really. Let's see why.

We can visualize two classes of students with the same mean exam score of 61. We will use a histogram of counts – it shows all observed exam scores on x-axis and their frequencies on y-axis:

Class A



Class B



In Class A, most students scored in the middle range (55-75). This suggests the class was challenging, but students were generally able to grasp the material.

In Class B, students either failed (scoring below 40) or excelled (scoring above 75), with almost no one scoring near the average. Since this is a university-level course, we would expect students to have the necessary background and motivation to achieve at least moderate success. The absence of middle-range scores suggests potential

problems with how the course was taught or assessed. Perhaps the material had a steep learning curve, or the assessment didn't align well with instruction.

Overall, despite same average score Class A's results appear more balanced and credible, suggesting better teaching effectiveness.

Measuring Spread of the Data

Our goal is to assess how student performance varies from one student to another. We use visualization to see this pattern, but we can compute a single number that captures the same information. This number is called variance, and it measures the dispersion of data, in other words, how much our observations differ from the mean value on average.

- Small variance indicates that most values are around the mean
- Large variance indicates the dataset contains values far from the mean – extreme values

Before diving deeper, let's establish some terminology:

- Population – all observation that exist of a specific type. Examples: All humans on Earth, all active students at the University of Toronto.
- Sample – one or more observation drawn from the population. Examples: 5000 random people, 400 random students from University of Toronto.

The key difference is that a population includes all possible observations of a certain kind, while a sample includes only some.

In practice, we usually want to infer information about a population using a sample because collecting data on the entire population is rarely feasible. For example, not all people who visit a restaurant (the population) leave a review – only some do (the sample) but we still draw conclusions based on rating calculated from that sample.

What makes a sample reliable? Think about restaurant ratings. We're more likely to trust a restaurant with 3,000 reviews and a 4.6-star rating than one with 30 reviews and a 4.7-star rating. Why? The larger sample size gives us more confidence that the rating reflects the true quality. The same principle applies across statistics – we need a sufficiently large sample to reliably represent the population.

Notation:

- n – sample size (30 students in our class).
- x_i – the i-th observation in our sample, where $i \leq n$ (represents score of a student with index i).
- $\bar{x}$ – sample mean or average, this is a sum of all observations in a sample divided by the sample size (e.g. mean score on the final exam). Formula to compute sample mean is as follows:

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$$

With the terminology in place, we can measure spread in the data with a single number. Intuitively, spread is how much observations tend to deviate from the mean. It is exactly what we will measure – average squared difference of all observations from the mean. However, there's one caveat – instead of dividing sum of squared differences by the sample size, we divide it by the sample size minus one:

$$Variance = \frac{\sum_{i=1}^{n} (x_i - \bar{x})^2}{n - 1}$$

Fundamentally, this change doesn't impact the intuition – it's just a formality that makes estimation of the population variance based on a sample more precise.

We want to measure how far every observation is from mean on average why square the differences?

If we simply averaged the raw differences ($x_i - \bar{x}$), positive and negative deviations would cancel out, giving us zero regardless of how spread out the data actually is.

For example: Suppose our sample has 2 observations and sample mean is 5, consider two datasets satisfying this condition:

- Dataset 1: {10, 0}
- Dataset 2: {5, 5}

These two datasets are very different – the first one has a large spread and the second one has absolutely no spread.

Without squaring, we get the following variances:

- Dataset 1:

$$\frac{(10-5)+(0-5)}{2-1}=\frac{5-5}{1}=\frac{0}{1}=0$$

- Dataset 2:

$$\frac{(5-5)+(5-5)}{2-1}=\frac{0+0}{1}=0$$

Both give zero, suggesting they have the same spread, but they clearly don't.

Once we square the differences – use variance formula, we get:

- Dataset 1:

$$\frac{(10-5)^2+(0-5)^2}{2-1}=\frac{25+25}{1}=\frac{50}{1}=50$$

- Dataset 2:

$$\frac{(5-5)^2+(5-5)^2}{1}=\frac{0+0}{1}=0$$

Now we correctly identify that dataset 1 has a much larger spread.

As you can observe from the example above, variance gets large pretty fast. A more interpretable metric is standard deviation (SD), which is simply the square root of variance:

$$SD=\sqrt{Variance}=\sqrt{\frac{\sum_{i=1}^{n}(x_i-\bar{x})^2}{n-1}}$$

Back to our exam score example, after computing standard deviation using formula above, we get the following summary for each class:

- Class A: $\overline{x} = 61$ and $SD = 8$
- Class B: $\overline{x} = 61$ and $SD = 34$

These numbers tell a clear story: Class A has most students centered near the mean (low spread), while Class B has students scattered widely across the score range (high spread).

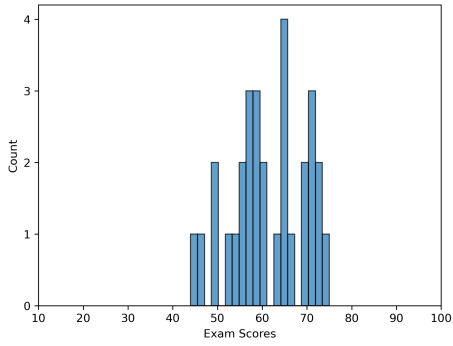
By knowing just two numbers – sample mean and standard deviation, we get a solid understanding of what the dataset looks like. These summary numbers are called statistics – single values computed from sample data that help us understand some characteristic of that data.

Probability

For a student entering Class A next term with the same professor, what grade should they expect to receive?

Obviously, this heavily depends on the individual student, but if we want to estimate a grade for a random student from the same university, we need to understand the likelihood of receiving each possible grade. One way to do this is to examine data from the previous term.

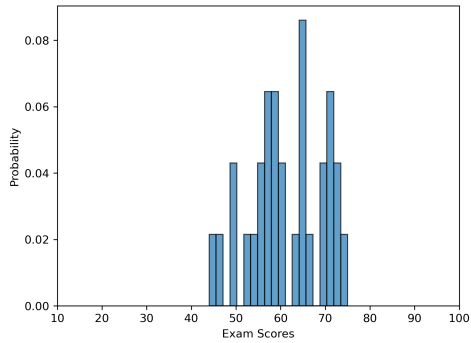
Let's plot the data for Class A:



We can see how many students received each grade. To convert this into probabilities, we simply divide each count by the total number of students (30). Probability of receiving a certain grade is essentially just a proportion of times this grade was received in our sample.

$$Probability\ of\ a\ grade = \frac{\#\ of\ times\ the\ grade\ was\ received}{Total\ \#\ of\ grades}$$

Let's plot the result:



Notice that the shape hasn't changed – only y-axis scale. Instead of counts, we now have proportions (or probabilities).

From this plot, we can see that a random student from that university has about 8% chance of scoring 65 and 4% chance of scoring 50. This way we can infer probability of receiving any score. Additionally, we can compute probability of receiving a score more or less than a certain value – by summing up probabilities of all scores above or below it respectively.

An important caveat: This analysis relies on a crucial assumption that our sample of 30 students is representative of all students who will take this class with this professor. In other words, we're assuming that future students will be similar to past students in terms of preparation, motivation, and ability.

How confident can we be in this assumption? As we discussed earlier, sample size is the key factor. A sample of 30 students is modest, meaning it gives us some insight, but we should be cautious about over-interpreting these exact probabilities. Ideally,

we'd want to collect data from multiple previous terms (perhaps 3-5 terms, giving us 90-150 students) to get more stable probability estimates.

What this means in practice: While we shouldn't treat these probabilities as exact predictions, they do provide useful ballpark estimates. A future student of Class A can reasonably expect that:

- Scores in the 60-70 range are fairly common (higher probabilities)
- Extreme scores (below 40 or above 85) are relatively rare (lower probabilities)

The overall distribution gives a sense of what's typical versus unusual in this class. Think of it like a restaurant with 30 reviews versus 300 reviews – both provide information, but you'd be more confident in conclusions drawn from the larger sample.

Distribution

A set of values with corresponding probabilities is called a probability distribution. Why is this useful?

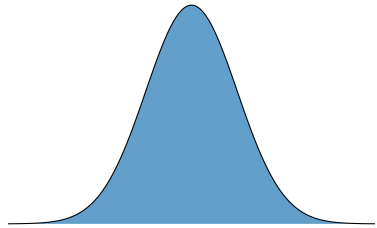
While a histogram of counts is specific to our sample of 30 students, a probability distribution is more general. It's not tied to a specific measurement type or sample size. Instead, it represents the underlying pattern of how values are distributed in the population.

This generality is powerful – a single type of probability distribution can represent many unrelated phenomena, such as:

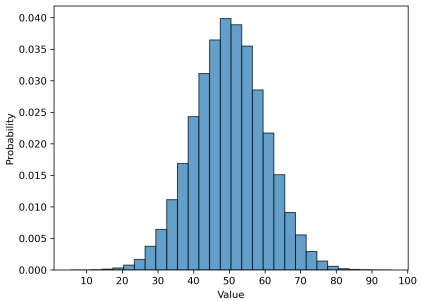
- Height distribution among NBA players
- Age distribution of first-time homebuyers worldwide
- Lifespan distribution of clownfish

Since certain patterns appear repeatedly in nature, statisticians have identified and named several "famous" probability distributions. Each of those distributions has some useful properties that allow us to infer useful information from data that follows that distribution.

We'll explore the most important one – normal distribution (also called the Gaussian distribution or bell curve). The general shape of this distribution looks as follows:



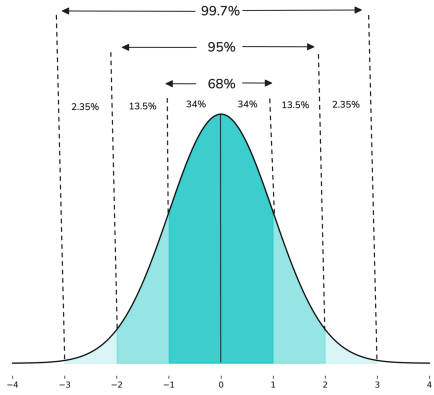
In practice, if we drew a large sample from a population that follows a normal distribution, it would look as follows:



What are properties of this distribution?

1. Symmetry: The distribution is perfectly symmetric around its center. The mean, median, and mode are all equal.
2. The 50-50 split: The mean divides the data in half – 50% of values fall below the mean, and 50% fall above it (directly related to symmetry).
3. The Empirical Rule (68-95-99.7). By knowing mean (M) and standard deviation (SD), we can estimate percentage of data points that lie within certain intervals:
    - 68% of data points lie in the interval $[M - SD, M + SD]$
    - 95% of data points lie in the interval $[M - 2 * SD, M + 2 * SD]$
    - 99.7% of data points lie in the interval $[M - 3 * SD, M + 3 * SD]$

This pattern can be easily observed through a normal distribution with mean 0 and standard deviation of 1:



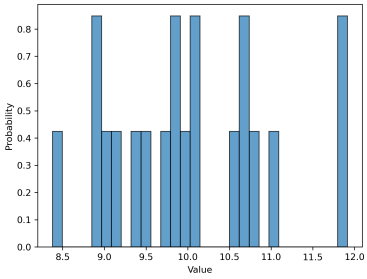
In fact, normal distribution is completely defined by just two numbers – its mean (*M*) and standard deviation (*SD*). These are called parameters of the distribution.

The type of a distribution and its parameters is all we need to generate data that follows the distribution. In fact, we can generate infinitely many of different normal distributions by varying values of mean and standard deviation.

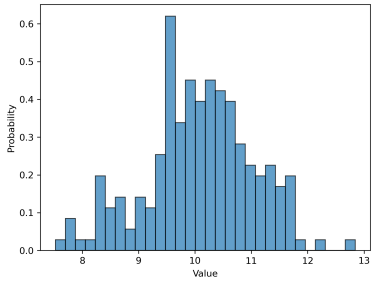
In practice, we are not given a distribution, just raw data – it's our job to decide whether it follows a certain "famous" distribution or not. In order to safely assume a distribution, we need a large sample size.

To illustrate why sample size matters – let's sample from the same distribution and plot histogram after 20, 200, 2 thousand and 20 thousand samples.

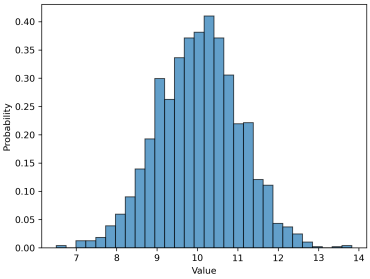
$n = 20$



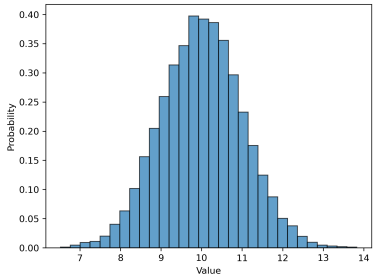
$n = 200$



n=2,000



n=20,000



- $n = 20$: Doesn't look normal
- $n = 200$: Starting to show bell shape
- $n = 2{,}000$: Clear bell curve emerges
- $n = 20{,}000$: Smooth, nearly perfect bell curve

Conclude: Even data drawn from a perfectly normal population won't look normal with a small sample size. The underlying pattern only becomes clear as sample size increases. This is why we should be cautious about drawing strong conclusions from small datasets.

Now let's come back to the initial questions and answer them using the information we learned in this chapter.

For illustration purposes, we will answer these questions for Class A.

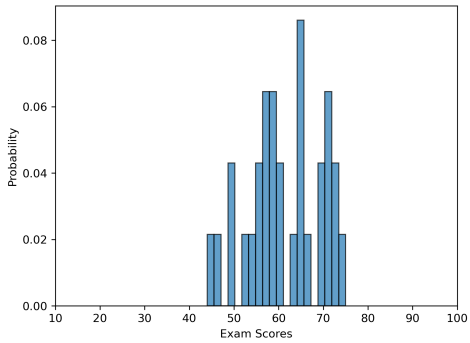
1. How well did the class of 30 students learn the material?
2. Did the professor do a good job teaching this class?
3. What grade should a future student in this class expect?

Since we can only infer information from the grades of these 30 students, we must make several strong assumptions:

- These 30 students represent the broader population of students who take this class
- The professor will teach similarly in future terms
- Course content and difficulty will remain stable
- No major external factors will change between terms

These are strong assumptions, which means we should not be overly confident in our conclusions. However, with these caveats in mind, it's reasonable to expect that future classes will perform similarly, making questions 1 and 3 essentially equivalent.

Before answering questions – let's take another look at the data of interest:



Question 1 and 3:

The typical score is around 61 (the mean), which indicates that on average students tend to pass this class.

Standard deviation is equal to 8. While the sample size of 30 is too small to reliably assume normality – by looking at the histogram, we can see that most data is centered around the mean.

Therefore, it is safe to say that most data points lie within one standard deviation from the mean, that is within this interval: $[61 - 8, 61 + 8] = [53, 69]$. This indicates that most people tend to achieve at least moderate success.

> Note: If we were confident the data followed a normal distribution, we could use 68-95-99.7 rule and say that this interval contains approximately 68% of student scores. Without that formal assumption, but observing that the data clusters around the mean, we can still reasonably say "most students" fall in this range. In practice, we can loosely apply the intuition from the 68-95-99.7 rule even when we haven't formally verified normality, as long as we see similar clustering patterns.

Question 2:

In question 1 and 2, we determined that most scores lie within $[53, 69]$ interval. Paired with the fact that histogram shows a relatively even distribution without extreme values – the teacher has done a decent job teaching the class as most students achieved at least moderate success.

In this module you got a gist of how we can leverage statistics to answer interesting questions about data.

Learning From Data (Optimization)

In this module, you will learn about the backbone of a great number of ML algorithms. We will derive the desired approach intuitively by solving a very simple problem.

The Language of Machine Learning (Functions)

The concept of a function is used throughout ML, so let's make sure you're comfortable with it first.

A function can be casually defined as a black box that requires some input and always produces output. Crucially, for the same input, a function must always produce the same output.

Let's look at an example:

$$y = f(x) = x + 5$$

In this case, y value depends on x; therefore, it is referred to as dependent variable. On the other hand, x doesn't depend on anything and thus is referred to as independent variable.

A function consists of exactly one dependent (output) and one or more independent variables (inputs). Here are some examples of functions with multiple independent variables:

$$y = r(x_1, x_2) = x_1 + x_2$$

$$y = k(x_1, x_2, x_3) = 7x_3 * (x_1^2 - \frac{x_2}{2})$$

$$y = z(x_1, x_2, x_3, x_4, x_5) = \frac{9x_1}{e^{x_5^{11}}}$$

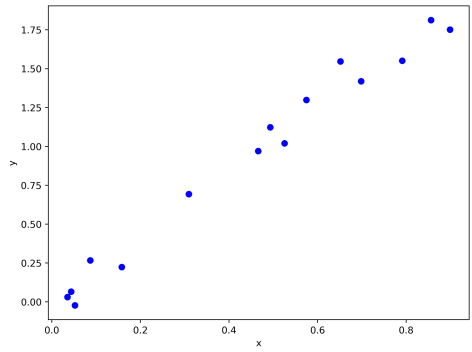
Get comfortable with the fact that a function can take many inputs, perform any kind of transformation and produce exactly one output.

Let's solve a simple problem where we want to make predictions about future based on historical data.

Our dataset:

	x	y
0	0.791320	1.551080
1	0.035284	0.030772
2	0.525270	1.020041
3	0.856293	1.812797
4	0.043562	0.065656
5	0.651788	1.546606
6	0.493195	1.122662
7	0.899229	1.750989
8	0.086825	0.266201
9	0.309275	0.692684
10	0.698256	1.419231
11	0.465975	0.970432
12	0.051959	-0.022401
13	0.574756	1.298906
14	0.157858	0.223998

Plotted:



Before diving in, we need to understand the data by answering two critical questions:

1. What are we trying to predict?
2. What will we use to predict that?

In the real world, a clear answer to both questions is always the most important part of the problem statement.

For this dataset, we want to predict y based on x. Looking at the graph, we can see that x and y have a positive linear relationship – as x increases, y increases at roughly the same rate.

> Note: Predicting one value based on another means figuring out the pattern of their relationship. We can formalize this pattern using a function.

Based on our observation, a linear function seems like a good fit:

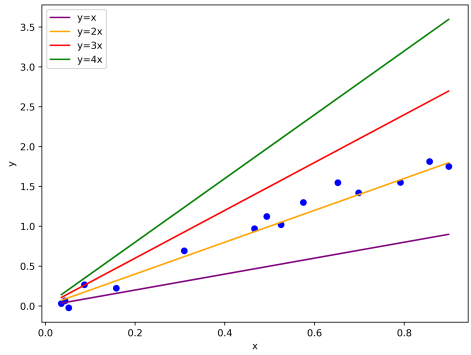
$$y = f(x) = \omega * x$$

Where:

- x – is a feature (input) from a single data point
- ω – is a scalar we need to find

This simple model can solve many different problems because the prediction pattern changes as ω changes. ω is a learnable parameter – it makes our ML algorithm generic. We'll soon discover why it's called "learnable."

Let's observe what kind of models we can potentially create with different values for ω:



Any of these models could be used to predict y for any value of x. From the graph, we can see that $y = 2x$ fits the data best, so $\omega = 2$ appears optimal.

Our model is done!

Now let's use our model to make a prediction for a new data point. For example, when $x = 0.6$:

$$y = 2 * 0.6 = 1.2$$

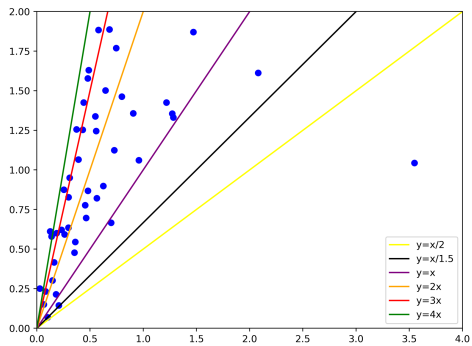
Great, we just made our first prediction!

The Problem With "Eyeballing It"

The intuition is good, but this approach has major issues for real-world problems. In practice:

- We have thousands or millions of observations
- The relationships are more complex
- There are usually many features (not just one x) and as many parameters

Let's try the same approach with a slightly more complex dataset:



Now it's unclear which value of ω is best or if it's even among the ones we plotted. Keep in mind, we're still dealing with only one feature. Imagine trying this with:

- Thousands or millions of data points
- Multiple features (3+ features are impossible to visualize)
- More complex relationships

It would be simply impossible.

Conclusion: After choosing a general model structure, we need a quantitative approach to find the optimal parameter value ω.

First, we need to understand, what "optimal" means.

- Optimal value – the parameter value that achieves our objective.
- Our objective – create the most accurate model. In other words, one that predicts values as closely to actual values as possible and learns the general pattern rather than memorizes training data.

To optimize is to find the optimal solution to our objective. Hence, optimization in this case refers to the process of finding learnable parameter value that achieves the objective.

To create a model is to train it, if we want to excel at this task – we need to understand how ML algorithms learn. Now, let's come back to student analogy and understand how they actually learn.

Training a Student

Until a student learned the pattern, for every problem in the book:

1. Take the problem description
2. Solve the problem with current knowledge
3. Compare the solution to the correct answer
4. Adjust approach based on the feedback

Training an ML Model

Until a model learned the pattern, for every data point in the dataset:

1. Take its feature
2. Make a prediction
3. Compare to the target
4. Adjust the current approach based on the feedback

Now let's formalize steps 2, 3, and 4 to train our model.

Step 2 – Making a Prediction

Recall, our model has the following general form:

$$y = \omega * x$$

There are two parts to the prediction – actual data point (x) and learnable parameter (ω). Data is constant; therefore, the only way to change prediction is through the learnable parameter.

The goal of the training process for this ML algorithm is to learn (find) value of this parameter that results into the most accurate predictions. But what happens when it makes its first prediction? It makes a guess. Hence, at the beginning of training we just assign a random value to ω.

Step 3 – Comparing to the Target

Every time the model makes a prediction, it needs feedback to learn. We formalize this using a loss function that computes how far the prediction is from the truth. The smaller the loss – the better the prediction.

Depending on a problem, we may prefer to use different loss functions – the one used in this sub-module is the most common loss for predicting a continuous target.

Loss function – error for a single observation, denoted by l:

$$l(y_{pred_i}, y_{actual_i}) = \left(y_{pred_i} - y_{actual_i}\right)^2$$

In words: Loss function in this case is squared difference between the predicted and actual target value for any single data point.

The loss function gives feedback about a single prediction. But if we want the model to receive feedback on multiple observations, we need to compute the average loss across a batch of data.

Cost function – average loss across all observations, denoted by *J*:

$$J(y_{pred}, y_{actual}) = \frac{1}{n} * \sum_{i=1}^{n} l(y_{pred_i}, y_{actual_i})$$

Equivalently:

$$J(y_{pred}, y_{actual}) = \frac{1}{n} * \sum_{i=1}^{n} (y_{pred_i} - y_{actual_i})^2$$

Cost function is like a student solving n problems without looking at solutions, then receiving an average score for all n problems. This particular cost function measures mean squared error (MSE). Different cost functions can be thought of as different grading system.

Step 4 – Learning From Feedback

That's where the quantitative part begins – actual learning from the feedback.

First, recall our prediction equation:

$$y_{pred} = \omega * x$$

Let's rewrite the cost function by substituting in our prediction equation:

$$J(\omega * x, y_{actual}) = \frac{1}{n} * \sum_{i=1}^{n} (\omega * x_i - y_{actual_i})^2$$

Notice, all values are given except ω. Therefore, the cost function depends only on ω. Recall, it's our objective to minimize the cost function – equivalently, make the most accurate prediction. Thus, we need to find ω that makes J as small as possible.

Let's code up the cost function defined earlier.

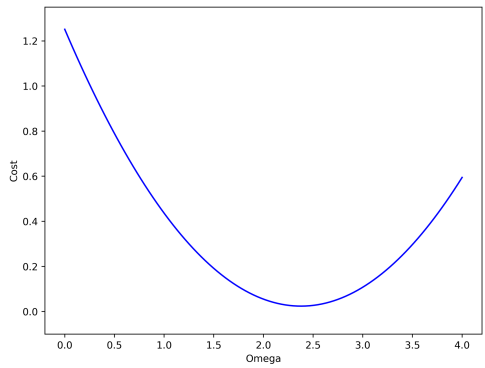
```
def cost(omega, x, y_actual):

    n = len(x)

    return (1 / n) * sum((omega * x - y_actual) ** 2)
```

Now, we can check how the cost function changes with different values of ω.

```
import numpy as np

omega_values = np.linspace(0, 4, 100)

cost_values = [cost(omega, x, y) for omega in omega_values]
```

Code above generates 100 omega values in the range from 0 to 4 and then computes cost function value for each one of them. Note that this is just an illustration to show the concept – you don't need to implement this yourself.

Let's plot the cost function value for different values of omega:



This plot shows how value of the cost J changes as learnable parameter ω changes. It allows us to visually identify the optimal value for ω – the one that makes cost the

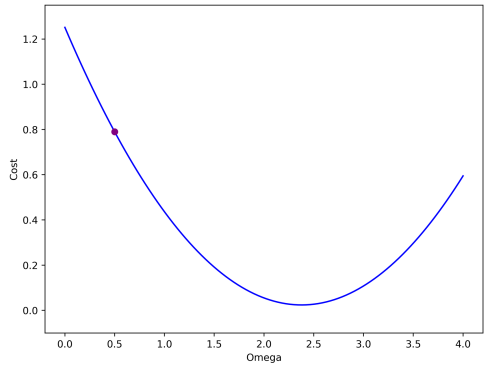
smallest, which is somewhere between 2 and 3. We could find the minimum value in "cost_values" and track the corresponding ω in "omega_values".

This approach works for this simple problem, but it doesn't scale – in the real world, we have many learnable parameters that affect J. Since it's not even possible to produce this plot for two or more learnable parameters, we conclude this approach is not general enough. Similarly, "cost_values" array becomes too large and expensive to construct very fast.

We need a better approach that works for any number of learnable parameters.

Begin Learning

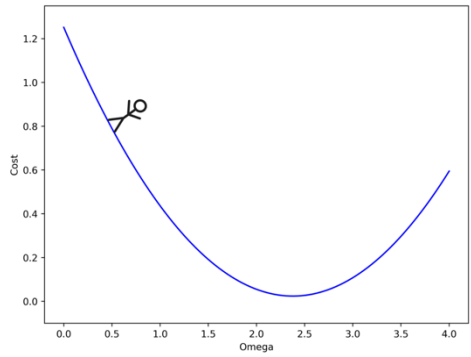
Our ML model begins by making a guess for the first prediction. In other words, we initialize value for the learnable parameter ω to be a random value, say 0.5. Let's represent the current omega with a purple circle and produce the plot:



> Note: In practice, we can't visualize this graph – at this point all information we have is about the location of the purple dot. In other words, all the model knows is that the cost function value is 0.8 when omega equals to 0.5.

Here's where it becomes fun!

Think of the learnable parameter as a person trying to walk down a hill, where the hill is the cost function. The purple dot is the person, and blue curve is the hill:



This person going down the hill is equivalent to us finding omega (ω) that minimizes the cost (maximizes accuracy) of the model.

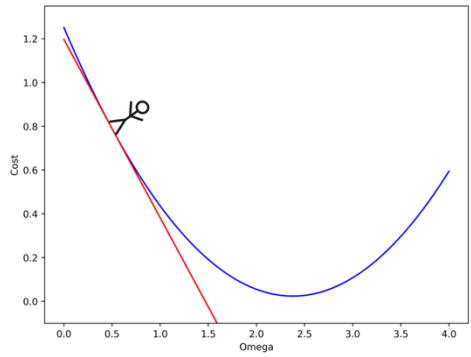
While a person can feel the ground (giving them information about direction and steepness), we don't have that information by default.

That's where calculus comes into play, giving our person the information they need – which direction to step and how far. In other words, whether to increase or decrease omega and by how much.

But how do we get that information?

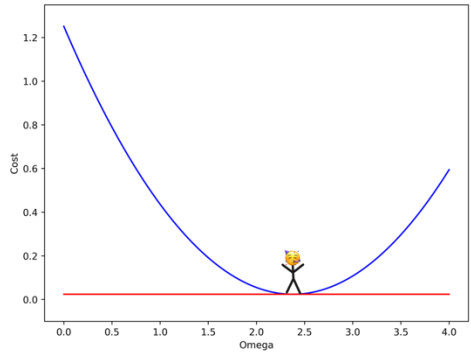
The good news: Every smooth function is locally linear, meaning if we zoom in close enough at any point, it looks like a straight line.

For instance, if we zoom in close enough at $\omega = 0.5$, the surface looks flat – just like a straight line. Extending that line reveals:



This means we can define a straight line at any point along the cost function. Intuitively, the slope of this line tells our person how steep the surface is and which way it tilts – which is exactly what we need.

Eventually, the goal is to go down the hill – in other words reach flat surface:



At this point, the slope of the straight line beneath our person is equal to 0.

To recap: A straight line (tangent) can be defined at any point on the cost function. The slope of that line guides us toward the flat surface where our objective is achieved.

Conclude the learning process:

1. Initialize omega to be random
2. Until we reach flat surface (slope equal to 0):
    - Compute slope of the straight line at the current point
    - Use it to inform whether we need to increase or decrease omega and by how much

The key is computing slope of the tangent line at every point and using it to inform how to change omega to minimize the cost. Now, let's observe how this movement downhill is manifested in math.

To find the slope of a line, we need two points. Recall that the slope between points (x_1, y_1) and (x_2, y_2) is:

$$\frac{y_2 - y_1}{x_2 - x_1}$$

Since y is a function of x, we can rewrite equation as:

$$\frac{f(x_2) - f(x_1)}{x_2 - x_1}$$

To find the slope at a specific point, we need two points that are infinitely close to each other. Let's say the distance between them is h, a tiny number approaching 0. Then we use the formula above to find the slope of straight line between two infinitely close points.

Two infinitely close points with distance h between them are: $(x, f(x))$ and $(x + h, f(x + h))$. Thus, the slope of a line defined by these two points is:

$$\frac{f(x + h) - f(x)}{(x + h) - x} = \frac{f(x + h) - f(x)}{h}$$

We can denote that h is infinitely close to 0 using limit notation:

$$\lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Conclude: The equation above allows us to compute the slope of a straight (tangent) line at any point x to any smooth function f. The slope of the tangent line tells us how a tiny change in x affects f – using this information we can keep changing x to decrease f until slope becomes close to 0.

In our case, we want to figure out how to change ω to make J as small as possible. J is at its minimum value when bottom of the hill is reached, which is indicated by slope of the tangent line being close to 0 (flat surface).

Using the slope formula, we can find how change in ω affects J:

$$\lim_{h \to 0} \frac{J(\omega+h) - J(\omega)}{h}$$

This quantity is also called derivative of J with respect to ω – those terms are equivalent. Hence, above equation can also be denoted as:

$$\frac{dJ}{d\omega} = \lim_{h \to 0} \frac{J(\omega+h) - J(\omega)}{h}$$

This gives our person information about the surface they're standing on. They use it to decide which way to step and how far.

Starting from their current ω, they keep taking steps guided by the slope until it reaches 0 – the bottom of the hill!

The Learning Algorithm

Let's formalize what we just learned purely in ML context:

1. Select random omega
2. Compute slope of the tangent line, that is $\frac{dJ}{d\omega}$
3. Until slope of the tangent line is around 0, that is $\frac{dJ}{d\omega} \approx 0$:
   - Update omega using slope, that is $\omega = \omega - \frac{dJ}{d\omega}$
   - Compute slope of tangent line at newly updated omega, that is $\frac{dJ}{d\omega}$
   - Repeat (3)

Eventually, slope becomes 0, which indicates minimum of the cost was reached and we learned optimal value for omega – model has been trained!

Now that we know how the algorithm works, let's compute slope of the tangent line of our cost function:

$$\frac{dJ}{d\omega} = \lim_{h \to 0} \frac{J(\omega) - J(\omega + h)}{h} =$$

$$\lim_{h \to 0} \frac{\sum_{i=1}^{n} \left[\left(\omega * x_i - y_{actual_i} \right)^2 - \left((\omega + h) * x_i - y_{actual_i} \right)^2 \right]}{h * n}$$

It may look scary, but we just used definition of the cost function. You are encouraged to revisit definition of J to make sure you fully understand the expression above.

Let's simplify the expression above, it's not necessary to go through this process as it's nothing but tedious math. If we expand the squared terms and perform some algebra, we get the following form:

$$\frac{dJ}{d\omega} = \frac{2}{n} * \sum_{i=1}^{n} x_i * \left(\omega * x_i - y_{actual_i} \right)$$

In code, this is done as follows.

```
def cost_derivative(omega, x, y_actual):

    n = len(x)

    return (2 / n) * sum(x * (omega * x - y_actual))
```

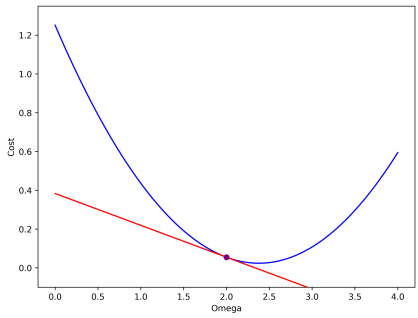
Now we can implement the full algorithm.

```
import random

omega = random.uniform(0, 4)
slope = cost_derivative(omega, x, y)

while abs(slope) > 0.0001:
    omega = omega - slope
    slope = cost_derivative(omega, x, y)

print(omega)

2.083087164799504
```

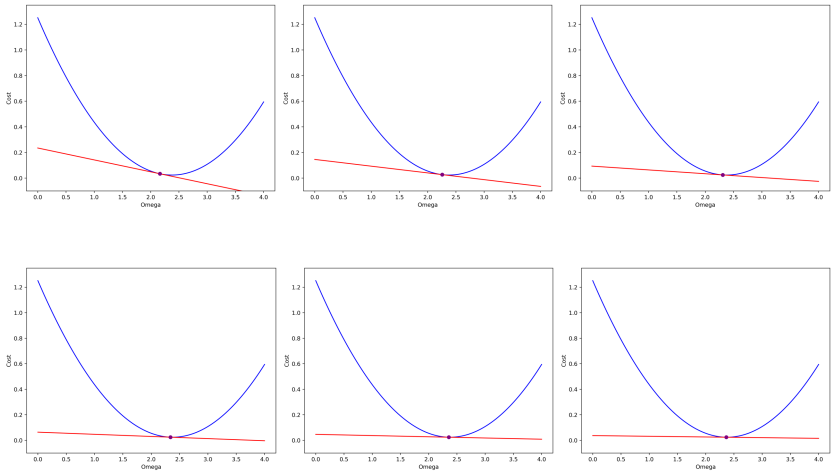
Note: Each line of code (aside from import and print) represents a step we defined earlier in our learning algorithm. To recap, we initialize omega, compute the slope, then keep updating omega using the slope until it gets close to 0. Finally, we reach our objective and print the optimal value for omega.

Let's illustrate what is happening after each iteration of this learning algorithm using our cost-omega graph.

The starting point (random omega):



After each iteration:



This illustrates how after each iteration we get closer and closer to the bottom of the hill until we reach it. The slope of the red line is what guides our next move at every iteration until eventually it becomes 0.

You now understand how the most important training algorithm in ML works! This core idea powers countless algorithms, including large language models (LLMs).

There are two more modifications that we need to consider making this algorithm more robust. In the original explanation, we keep training the algorithm until slope approaches 0. While it's the ideal state we want to reach – in practice training algorithms on a lot of data becomes computationally expensive. In order to finish training within a reasonable time and computational constraint – algorithms are trained for a set number of iterations. Simply put, instead of a while loop, we use for loop to restrict training expanses. The number of iterations needs to be large enough for the algorithm to converge, that is for the slope to become relatively small.

We know that information from the slope (derivative) provides learnable parameter with direction and magnitude of its next step. What if we were to introduce a variable to control size of each step?

This allows us to control how fast the person gets down the hill, which directly relates to the number of steps they need to make to reach the bottom (number of iterations).

This new variable provides us with more flexibility, allowing to make training faster and more efficient. The variable can be thought of as step size and is formally defined as learning rate denoted by alpha (α).

In essence, we scale derivative by alpha to control step size on each iteration:

$$\omega = \omega - \alpha * \frac{dJ}{d\omega}$$

Let's update the learning algorithm with these two changes.

```
num_iterations = 100
alpha = 0.1

omega = random.randint(0, 4)
slope = cost_derivative(omega, x, y)

for _ in range(num_iterations):

    omega = omega - alpha * slope
    slope = cost_derivative(omega, x, y)

print(omega)

2.077084848143789
```

We replaced while loop with a for loop and introduced alpha to control step size on each iteration – as the result we get an identical result to the previous version of this algorithm. What's important is that this version scales with complexity of the real-world problems.

The number of iterations and step size are values that interfere with how an ML algorithm works and are set by ML professionals, such quantities are referred to as hyperparameters.

The next logical question is: How to select appropriate values for the hyperparameters?

While there's no ultimate answer to this question as it depends on the problem, size and complexity of the dataset – we will explore how professionals approach this.

The Number of Iterations

The number of iterations is equivalent to the number of times learnable parameter gets updated. There's no point updating ω when the model stops making substantial progress, that is when the slope becomes very small (nearly flat surface).

Good starting points in the real world are 1,000 or 5,000 iterations, depending on dataset size and complexity.

How to know if it's right? Monitor the cost as training progresses:

- Too small: Algorithm stops before the model hits diminishing returns
- Too big: Algorithm runs long after progress stops
- Just right: Algorithm stops when cost stops changing significantly.

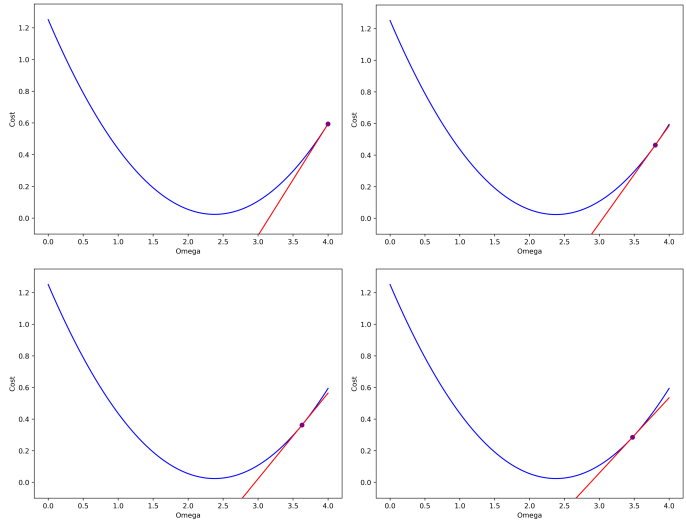
If the number is too small, the ML model will underperform as parameters never become optimal. If it's too large, performance usually won't decline – we are just wasting time and resources by performing unnecessary computations. You will get more understanding of this as we move forward.

Learning Rate

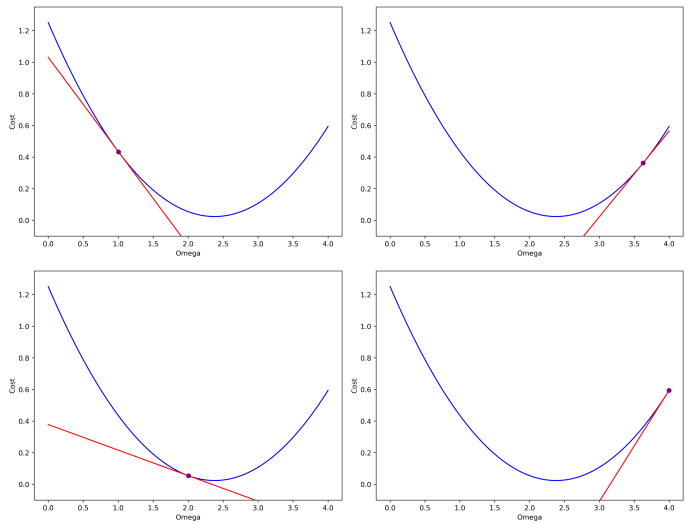
Step size is crucial for robust learning process and what is too big and too small is best shown visually.

Good starting points in the real world are 0.01, 0.001 or 0.0001, depending on the size and complexity of the dataset. We will use our learning algorithm with number of iterations set to 1000. To illustrate learning process under different step sizes, we will plot cost-omega graph at the beginning and after each 10 iterations of learning.

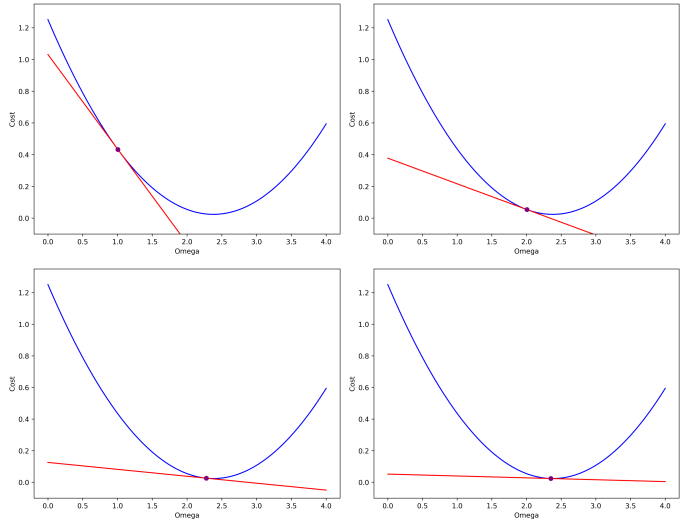
$\alpha = 0.001$ – too small:



$\alpha = 5$ – too big:



$\alpha = 0.01$ – just right:



How do you know whether it's too big, too small or just right without the visual? Think about what the visuals tell us. Learning rate is too small when value of the cost function doesn't change much starting from the very beginning. Alpha is too big when value of the cost function sometimes becomes substantially bigger as the model is being trained (opposite of what we want), meaning value of the cost is erratic. In both cases we never achieve optimal learnable parameter value. The value is just right when model makes substantial incremental progress towards the minimal value of the cost function.

To recap: The number of iterations and learning rate account for the number of steps and step size of the learnable parameter towards minimal value of the cost function. They are also referred to as hyperparameters of a model.

Multiple Learnable Parameters

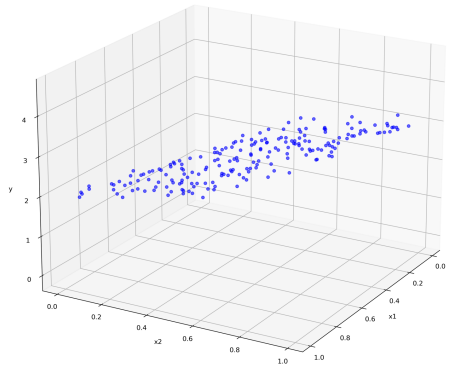
So far, we have explored how to train a linear model with a single learnable parameter. Let's expand to multiple learnable parameters. The core ideas remain the same.

The new dataset with multiple features:

	x1	x2	y
0	0.374540	0.642032	2.805723
1	0.950714	0.084140	2.155949
2	0.731994	0.161629	2.017069
3	0.598658	0.898554	3.861953
4	0.156019	0.606429	2.163741
...	...	...	...
195	0.349210	0.930757	3.407596
196	0.725956	0.858413	4.054195
197	0.897110	0.428994	3.076179
198	0.887086	0.750871	4.002891
199	0.779876	0.754543	3.732623

200 rows × 3 columns

Plotted:



We'll still use a linear model, but now we predict y based on two features: x_1 and x_2. Each feature provides different information and has different influence on the target. Therefore, each feature gets its own "weight" – its own learnable parameter that represents its influence on the target.

Our new prediction equation becomes:

$$y = \omega_1 * x_1 + \omega_2 * x_2$$

Let's update the algorithm with these changes:

1. Select multiple random omegas
2. For a set number of iterations:
    - Compute slope of the tangent line for each omega
    - Update each omega using corresponding slope, scaled by learning rate

Our cost function remains the same, it still measures average squared difference between predicted and actual value:

$$J = \frac{1}{n} * \sum_{i=1}^{n} \left(y_{pred_i} - y_{actual_i}\right)^2$$

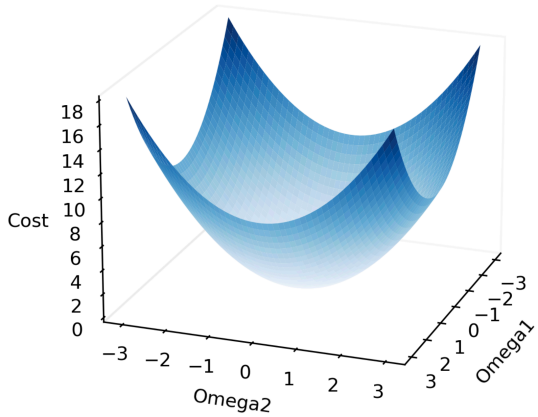
Since our y_{pred} has changed, it now involves two learnable parameters – J becomes dependent on two parameters, instead of just one:

$$J(\omega_1, \omega_2) = \frac{1}{n} * \sum_{i=1}^{n} \left(\omega_1 * x_{1_i} + \omega_2 * x_{2_i} - y_{actual_i}\right)^2$$

Because we have two parameters to tune, we need to calculate the slope (the derivative) for each one individually.

Think back to our hill analogy. Previously, our person was stuck on a single path and could only move forward or backward (increasing or decreasing a single ω value). But now, with two parameters, our person can move freely in any direction, not just forward or backward along a fixed path. The movement happens when we change parameter values for ω_1 and ω_2.

Here's what the cost surface looks like for two parameters:



While we can only visualize the hill in 2D or 3D, the beauty of this math is that it works for any number of parameters**.** Even if you have 100 or 1 million features, the logic remains the same: we find the slope for each individual parameter and take a small step downhill to minimize our error.

To find the way down, we look at the slope of the tangent line for the first parameter:

$$\frac{dJ}{d\omega_1} = \lim_{h \to 0} \frac{J(\omega_1 + h, \omega_2) - J(\omega_1, \omega_2)}{h}$$

And for the second:

$$\frac{dJ}{d\omega_2} = \lim_{h \to 0} \frac{J(\omega_1, \omega_2 + h) - J(\omega_1, \omega_2)}{h}$$

> Note: In each case, the other parameter stays the same – we are isolating how a tiny change in just one specific parameter affects the total cost J.

Finally, our person uses the information from the slope across each axis to take an informed step in the right direction:

$$\omega_1 = \omega_1 - \alpha * \frac{dJ}{d\omega_1}$$

$$\omega_2 = \omega_2 - \alpha * \frac{dJ}{d\omega_2}$$

Congratulations! You just learned how to train a linear model with multiple learnable parameters!

Matrix Notation

Whenever we deal with multiple quantities of the same kind, it is useful to consider whether they can be represented in a more comprehensive structure. In this case we have multiple features (x_1, x_2), learnable parameters (ω_1, ω_2) and derivatives $(\frac{dJ}{d\omega_1}, \frac{dJ}{d\omega_2})$. Each feature is a vector, while each parameter and derivative is a scalar. Therefore, we can create a matrix of features:

$$X = \begin{bmatrix} x_1 & x_2 \\ \vdots & \vdots \end{bmatrix}$$

A vector of learnable parameters:

$$\omega = \begin{bmatrix} \omega_1 \\ \omega_2 \end{bmatrix}$$

And a vector of derivatives with respect to each parameter:

$$\frac{dJ}{d\omega} = \begin{bmatrix} \frac{dJ}{d\omega_1} \\ \frac{dJ}{d\omega_2} \end{bmatrix}$$

Vector $\frac{dJ}{d\omega}$ represents how change in every learnable parameter affects cost J. It can be thought of as ultimate information to guide the person down the hill at any point

of their journey. This type of vector has a special name – the gradient, denoted by nabla symbol next to the cost:

$$\nabla J = \frac{dJ}{d\omega} = \begin{bmatrix} \frac{dJ}{d\omega_1} \\ \frac{dJ}{d\omega_2} \end{bmatrix}$$

With matrix notation, our learning rule becomes compact and scales to any number of parameters.

This equation can be used to make a prediction for datasets with arbitrary number of features:

$$y_{pred} = X \cdot \omega$$

On each iteration compute the gradient and update parameters:

$$\nabla J = \frac{2}{n} * X^T \cdot (y_{pred} - y_{actual})$$

$$\omega = \omega - \alpha * \nabla J$$

Note: If you're curious how this matrix form relates to the individual derivatives we derived earlier, you can verify that it is equivalent to computing each derivative separately and stacking the results into a vector. The matrix multiplication and transposition covered in the "Manipulating Data" chapter make this operation both efficient and general – this equation works whether you have 3 parameters or 3 million.

Finally, the concept of gradient is crucial in ML as it allows us to optimize parameters. In practice, the term gradient is loosely used to refer to any vector of derivatives of a single objective function.

Let's implement complete training algorithm and solve our problem.

```
def cost_gradient(omega, X, y_actual):

    n = len(X)

    return (2 / n) * X.T @ (X @ omega - y_actual)

num_params = X.shape[1]

num_iterations = 1000
alpha = 0.01

omega = np.array([random.randint(0, 4) for _ in range(num_params)])

for _ in range(num_iterations):

    gradient = cost_gradient(omega, X, y)
    omega = omega - alpha * gradient

print(omega)

x1    1.920969
x2    3.072803
```

First, we defined a function to compute the gradient, then we identified the number of parameters (equal to the number of features) and finally trained the model using our learning algorithm.

At this point the model learned optimal values for ω and is ready to predict targets based on unseen features. In mathematical terms our model can be defined as follows:

$$y_{pred} = 1.92 * x_1 + 3.07 * x_2$$

Or in matrix form as:

$$y_{pred} = X \cdot \begin{bmatrix} 1.92 \\ 3.07 \end{bmatrix}$$

This training algorithm that we learned is called gradient descent because the gradient of the cost function guides the parameters to progressively descend toward the minimum cost. Gradient descent is an optimization technique, as it seeks the optimal values of learnable parameters.

This algorithm or its variations power everything from simple linear models to massive neural networks.

Congratulations! You've just learned how the most fundamental ML algorithms actually learn from data!

You've seen the foundation.

232 pages of building core ML to go!

Linear Regression · Logistic Regression · Regularization
K-Nearest Neighbors · Naïve Bayes · Decision Tree
Random Forest · XGBoost · Neural Network

Continue reading on Amazon

Search: Machine Learning From Scratch, Akimenko